Part 4：OpenClaw 热门 Skills 实操

> *52 个内置技能、13,000+ 社区扩展——今天我们亲手激活其中最强的 5 个，看看 AI 助手"真正干活"是什么体验。*

引言

装了 OpenClaw 却不装 Skill，就像买了一台顶配 iPhone 却一个 App 都不装——你拥有了世界上最强大的硬件，但只能用来打电话。

前三章，我们把 OpenClaw 装好了、配好了、保护好了。它已经能和你对话了，能回答问题，能执行简单的文件操作。但说实话，到目前为止，这些事 ChatGPT 也能做。一个"能聊天的 AI"和一个"能干活的 AI 助手"之间的鸿沟，靠什么来填补？答案就是 **Skill**。

Skill（技能）是 OpenClaw 的 App Store。截至 2026 年 3 月，社区注册表 ClawHub 上已经有 **13,729 个 Skill**，覆盖浏览器自动化、搜索研究、文件处理、数据分析、代码执行等 **18 个大类**。从帮你读邮件到控制智能家居灯泡，从自动写代码到分析股票数据——只要你能想到的，大概率有人已经做了一个 Skill。

今天，我精心挑选了 5 个最实用、最酷炫的 Skill，设计了 5 个原创演示场景，带你亲眼看看 AI 助手"真正干活"是什么样子的。看完这一章，你会对 OpenClaw 的能力边界有一个全新的认识。

4.1 Skill 概念介绍与 ClawHub 官网介绍

在动手安装 Skill 之前，我们先花几分钟理清几个核心概念。这些概念不难，但理解清楚了，后面遇到问题你才知道从哪里下手排查。

Tools vs Skills：器官 vs 教科书

第一个容易混淆的概念：**Tools 和 Skills 到底有什么区别？**

一句话概括：**Tools 是器官，Skills 是教科书。**

维度	TOOLS（工具）	SKILLS（技能）
类比	器官——决定 OpenClaw 能做什么	教科书——教 OpenClaw 怎么做
本质	底层能力接口（读写文件、执行命令、浏览网页）	基于 SKILL.md 的工作流指令（编排多个 Tool 完成任务）
形式	代码级 API，由 Gateway 内核提供	Markdown 文件 + YAML 元数据头
管理方式	`tools.allow` / `tools.deny` 全局开关	`skills.entries` 按名称启用/禁用
例子	`read`、`write`、`exec`、`browser`	`coding-agent`、`weather`、`tavily`

打个具体的比方。`read` 工具就像眼睛——它让 OpenClaw 能"看"到文件内容。`write` 工具就像手——它让 OpenClaw 能"写"入文件。`exec` 工具就像大脑的执行区——它让 OpenClaw 能"运行"命令。`browser` 工具就像脚——它让 OpenClaw 能"走"进互联网。这些都是 **Tools**，定义了 OpenClaw 的"感官"和"肢体"。

而 `coding-agent` Skill 呢？它就像一本《软件工程教材》。这本教材告诉 OpenClaw："当用户让你写代码时，你应该先用 `read` 工具读懂现有代码结构，再用 `write` 工具创建新文件，然后用 `exec` 工具运行测试来验证。如果测试失败了，再用 `read` 读取错误日志，用 `edit` 修改代码，反复迭代直到通过。" 这套**精心设计的工作流程**，就是 Skill 的核心价值。

> *核心关系：Skills 编排 Tools 来自动化复杂任务。一个 Skill 可能调用多个 Tool，按照预设的逻辑完成一项专业工作。没有 Tool，Skill 只是一纸空文；没有 Skill，Tool 只是散装零件。*

Skill 的技术本质：一个 SKILL.md 文件

每个 Skill 本质上就是一个目录，里面有一个核心文件叫 `SKILL.md`。这个文件分为两部分：

1. **YAML 头部**：声明 Skill 的元数据（名称、描述、依赖、平台要求）
2. **Markdown 正文**：用自然语言写的"使用说明书"——告诉 AI 什么时候用这个 Skill，怎么用

来看一个真实的例子：

```
---
name: openclaw-tavily-search
```

```
label: "Tavily 搜索"
description: "Web search via Tavily API..."
user-invocable: true
metadata:
  openclaw:
    emoji: "🔍"
    os: ["darwin", "linux", "win32"]
    requires:
      env: ["TAVILY_API_KEY"]
---

# Tavily 搜索

Use the Tavily API to search the web for current information.

## When to use
- User asks to search the web, look up information, or find
links
- User needs current/recent information beyond your training
data
...
```

看到了吗？`requires.env: ["TAVILY_API_KEY"]` 告诉 OpenClaw："要激活这个 Skill，必须先设置 `TAVILY_API_KEY` 环境变量。" 这就是为什么有些 Skill 显示为 "missing"——它们的依赖条件没有满足。

Skill 的三级加载体系

OpenClaw 从三个位置加载 Skills，按优先级从高到低排列：

```
┌──────────────────────────────────────────────────┐
│  ① Workspace Skills（最高优先级）                  │
│  路径：~/.openclaw/workspace/skills/              │
│  范围：当前 Agent 可见                            │
│  用途：从 ClawHub 安装的社区 Skill 放在这里        │
├──────────────────────────────────────────────────┤
│  ② Managed / Local Skills                        │
│  路径：~/.openclaw/skills/                        │
│  范围：本机所有 Agent 共享                        │
```

```
│  用途：手动创建的自定义 Skill 放在这里                    │
├──────────────────────────────────────────────────────┤
│  ③ Bundled Skills（最低优先级）                        │
│  路径：~/.npm-global/lib/node_modules/openclaw/      │
│       skills/                                        │
│  范围：随安装包自带（52 个内置 Skill）                   │
│  用途：OpenClaw 官方预置的技能                          │
└──────────────────────────────────────────────────────┘
```

冲突处理规则：如果同名 Skill 出现在多个层级，高优先级层覆盖低优先级层。这个设计很聪明——假设你觉得内置的 `weather` Skill 不够好用，完全可以在 Workspace 层放一个自定义版本，它会自动替代内置版本，不需要修改任何配置。

实际应用举例：

- **个人定制**：你写了一个定制版的 `summarize` Skill，放在 Workspace 层，只有当前 Agent 使用
- **团队共享**：运维团队做了一个公司专用的 `deploy` Skill，放在 Managed 层，公司所有 Agent 共享
- **开箱即用**：刚安装好 OpenClaw，Bundled 层已经有 52 个内置 Skill 可以用

Skill 就绪的判断标准

一个 Skill 被标记为 `✓ ready`（就绪）需要**同时满足**以下全部条件：

条件	说明	示例
二进制依赖就位	`requires.bins` 中声明的 CLI 工具存在于 `$PATH` 中	github Skill 需要 `gh`
环境变量已设置	`requires.env` 中声明的变量已配置	tavily Skill 需要 `TAVILY_API_KEY`
配置键已存在	`requires.config` 中声明的键在 openclaw.json 中	slack Skill 需要 `channels.slack`
平台匹配	`os` 字段包含当前操作系统	某些 Linux 专用 Skill 在 Mac 上不可用

缺一不可。这就是为什么我们的 Mac 上 52 个 Skill 只有 9 个就绪——大部分 Skill 的二进制依赖（比如 `gh`、`memo`、`nano-pdf` 这些命令行工具）还没有安装。

运行时注入机制

一个重要的技术细节：OpenClaw 在 **Gateway 启动时**（而不是对话时）快照当前就绪的 Skills 列表，将它们的指令注入到系统提示词中。后续对话轮次复用同一份列表。

这意味着什么？

1. 安装新 **Skill** 后必须重启 **Gateway**，否则新 Skill 不会被加载
2. **不要同时激活太多 Skill**——每个 Skill 大约消耗 24 tokens 的上下文窗口，建议控制在 4-5 个同时激活

ClawHub 注册表：13,729 个 Skill 等你挑

ClawHub（clawhub.com）是 OpenClaw 的公共 Skill 注册表，类似于 npm 之于 Node.js、PyPI 之于 Python——一个集中管理和分发 Skill 的平台。

来看一组核心数据：

指标	数据
总 Skill 数量	**13,729+**（截至 2026 年 2 月）
精选高质量 Skill	5,490+（经社区筛选）
主要分类	**18+** 个
开源协议	绝大多数 MIT / MIT-0

18 个分类中，最热门的几个：

分类	SKILL 数量	代表 SKILL
Coding & IDEs	1,222	coding-agent, code-review
Web & Frontend	938	前端开发工具集
DevOps & Cloud	409	github, gh-issues
Search & Research	352	tavily, web-search
Browser & Automation	335	agent-browser
Productivity & Tasks	206	mission-control
Git & GitHub	170	github, gh-issues
Image & Video	169	openai-image-gen
Communication	149	slack, discord
PDF & Documents	111	nano-pdf, summarize

可以看到，**Coding** 类占绝对大头（1,222 个），这说明 OpenClaw 的核心用户群是开发者。但不要以为它只能写代码——搜索、浏览器、文件处理、数据分析这些"非程序员"场景同样非常丰富。而且数量增长很快，每周都有新的 Skill 被开发者贡献到 ClawHub 上。

本机 Skill 现状：52 个内置，9 个就绪

回到我们自己的 Mac。在 Part 2 配置好 Gateway 之后，OpenClaw 已经自动发现了 52 个内置 Skill。但检查一下状态你会发现——只有 **9** 个是就绪状态（ready），其余 43 个都是缺失状态（missing）。

那 9 个就绪的 Skill 分别是：

SKILL	功能	为什么就绪
coding-agent	委派编码任务给 Codex/Claude Code	自动检测到本机有 claude/codex
healthcheck	安全加固和风险检查	无额外依赖
node-connect	诊断节点连接问题	无额外依赖
openai-image-gen	通过 OpenAI API 批量生成图片	无额外依赖（需 API Key 时才用）
openai-whisper-api	通过 Whisper API 语音转文字	无额外依赖
skill-creator	创建/编辑/审计 Skill	无额外依赖
tmux	远程控制 tmux 会话	系统已安装 tmux
video-frames	用 ffmpeg 从视频提取帧	系统已安装 ffmpeg
weather	天气查询（无需 API Key）	无额外依赖

其余 43 个 Skill 因缺少对应的 CLI 工具、环境变量或配置项而标记为 missing。比如 `github` Skill 需要 `gh` 命令行工具，`nano-pdf` Skill 需要 `nano-pdf` CLI，`slack` Skill 需要配置 Slack 频道 token。

这正是我们接下来要做的——**安装依赖，让更多 Skill 从 "missing" 变为 "ready"**。

4.2 Skill 管理命令使用方法

管理 Skill 有两套命令体系：**内置管理命令**和 **ClawHub CLI**。前者用于查看和诊断本机 Skill 状态，后者用于从社区搜索和安装新 Skill。掌握这两套命令，你就能完全掌控 OpenClaw 的 Skill 生态。

内置 Skills 管理命令

这些命令以 `openclaw skills` 开头，管理的是**本机已有的 Skill**：

命令	功能	典型用途
`openclaw skills list`	列出所有 Skill 及状态	快速概览全貌（52 个 Skill 一目了然）
`openclaw skills list --eligible`	仅列出就绪的 Skill	确认哪些 Skill 能用
`openclaw skills check`	汇总就绪/缺失统计	诊断问题——缺什么一看便知
`openclaw skills info <name>`	查看 Skill 详细信息	了解依赖要求和安装路径
`openclaw skills preview <name>`	预览 Skill 源码	安装前安全检查——看看它做了什么
`openclaw skills vet <name>`	运行安全扫描	自动检测恶意代码模式

ClawHub CLI 命令

这些命令以 `npx clawhub` 开头，操作的是**ClawHub 社区注册表**：

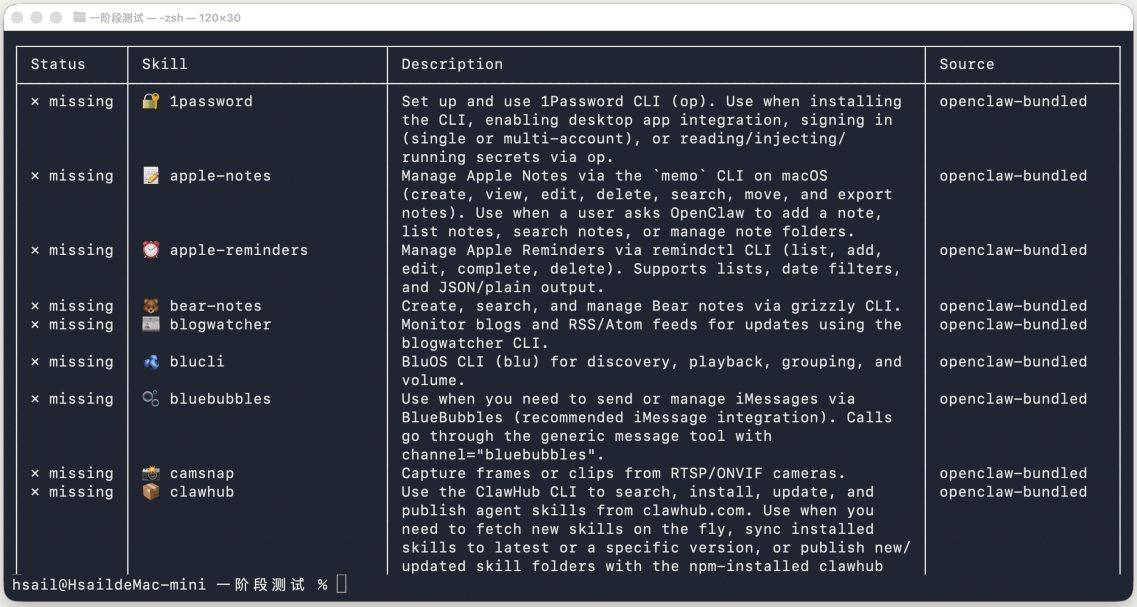
命令	功能	典型用途
`npx clawhub search <关键词>`	搜索 ClawHub 注册表	发现新的 Skill
`npx clawhub inspect <slug>`	查看详情（不安装）	了解作者、版本、许可证
`npx clawhub install <slug>`	安装 Skill 到本机	添加新能力
`npx clawhub update --all`	更新所有已安装的 Skill	保持最新版本

> *小知识：`npx` 是 Node.js 自带的命令执行工具。当你运行 `npx clawhub` 时，如果本地没有安装 clawhub CLI，npx 会自动下载并运行它。所以你不需要事先安装 clawhub——直接用 npx 调用即可。*

实操：查看当前 Skill 状态

理论讲够了，让我们实际操作一下。打开终端，运行第一条命令：

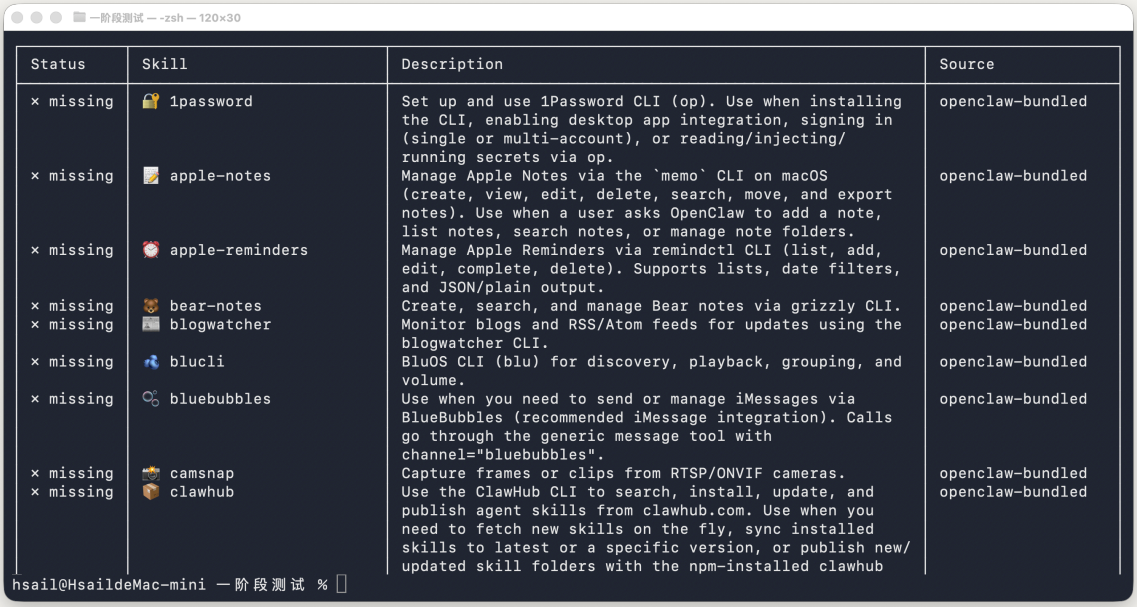
```
$ openclaw skills list
```



Status	Skill	Description	Source
× missing	1password	Set up and use 1Password CLI (op). Use when installing the CLI, enabling desktop app integration, signing in (single or multi-account), or reading/injecting/running secrets via op.	openclaw-bundled
× missing	apple-notes	Manage Apple Notes via the `memo` CLI on macOS (create, view, edit, delete, search, move, and export notes). Use when a user asks OpenClaw to add a note, list notes, search notes, or manage note folders.	openclaw-bundled
× missing	apple-reminders	Manage Apple Reminders via remindctl CLI (list, add, edit, complete, delete). Supports lists, date filters, and JSON/plain output.	openclaw-bundled
× missing	bear-notes	Create, search, and manage Bear notes via grizzly CLI.	openclaw-bundled
× missing	blogwatcher	Monitor blogs and RSS/Atom feeds for updates using the blogwatcher CLI.	openclaw-bundled
× missing	blucli	BluOS CLI (blu) for discovery, playback, grouping, and volume.	openclaw-bundled
× missing	bluebubbles	Use when you need to send or manage iMessages via BlueBubbles (recommended iMessage integration). Calls go through the generic message tool with channel="bluebubbles".	openclaw-bundled
× missing	camsnap	Capture frames or clips from RTSP/ONVIF cameras.	openclaw-bundled
× missing	clawhub	Use the ClawHub CLI to search, install, update, and publish agent skills from clawhub.com. Use when you need to fetch new skills on the fly, sync installed skills to latest or a specific version, or publish new/updated skill folders with the npm-installed clawhub	openclaw-bundled

按下回车，OpenClaw 会扫描三个层级的 Skill 目录（Workspace → Managed → Bundled）：

扫描完成后，你会看到一张完整的表格：



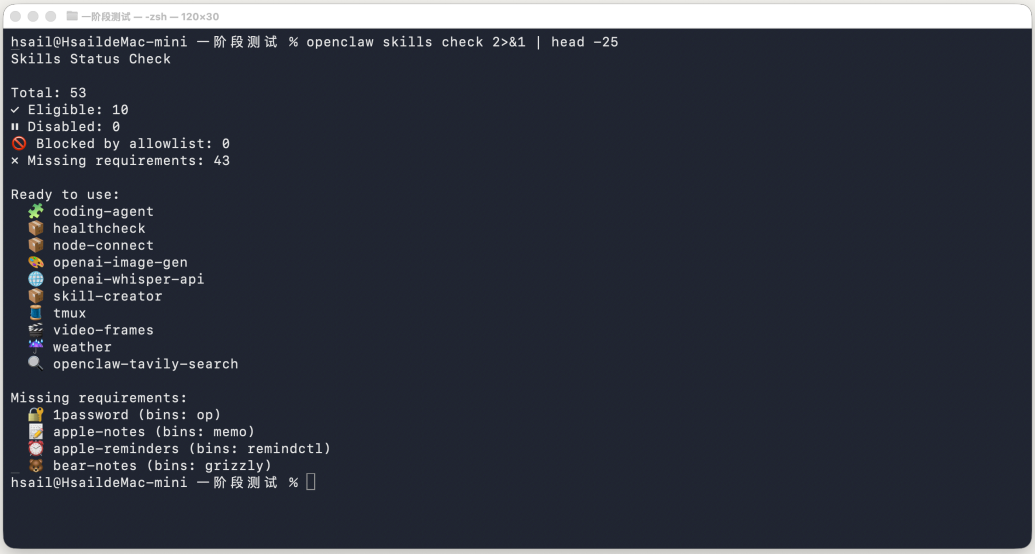
Status	Skill	Description	Source
× missing	1password	Set up and use 1Password CLI (op). Use when installing the CLI, enabling desktop app integration, signing in (single or multi-account), or reading/injecting/running secrets via op.	openclaw-bundled
× missing	apple-notes	Manage Apple Notes via the `memo` CLI on macOS (create, view, edit, delete, search, move, and export notes). Use when a user asks OpenClaw to add a note, list notes, search notes, or manage note folders.	openclaw-bundled
× missing	apple-reminders	Manage Apple Reminders via remindctl CLI (list, add, edit, complete, delete). Supports lists, date filters, and JSON/plain output.	openclaw-bundled
× missing	bear-notes	Create, search, and manage Bear notes via grizzly CLI.	openclaw-bundled
× missing	blogwatcher	Monitor blogs and RSS/Atom feeds for updates using the blogwatcher CLI.	openclaw-bundled
× missing	blucli	BluOS CLI (blu) for discovery, playback, grouping, and volume.	openclaw-bundled
× missing	bluebubbles	Use when you need to send or manage iMessages via BlueBubbles (recommended iMessage integration). Calls go through the generic message tool with channel="bluebubbles".	openclaw-bundled
× missing	camsnap	Capture frames or clips from RTSP/ONVIF cameras.	openclaw-bundled
× missing	clawhub	Use the ClawHub CLI to search, install, update, and publish agent skills from clawhub.com. Use when you need to fetch new skills on the fly, sync installed skills to latest or a specific version, or publish new/updated skill folders with the npm-installed clawhub	openclaw-bundled

表格最上方的 **Skills (9/52 ready)** 一目了然——52 个 Skill 中只有 9 个标记为绿色的 `✓ ready`，其余都是红色的 `✗ missing`。

注意表格中的 **Source** 列：所有 Skill 的来源都标记为 `openclaw-bundled`，说明这 52 个全部是随安装包自带的内置 Skill。后面我们从 ClawHub 安装的 Skill 会显示为 `openclaw-workspace`。

再来看看具体缺少什么。运行诊断命令：

```
$ openclaw skills check
```



这份报告非常有价值。**Missing requirements** 部分清楚列出了每个 Skill 缺少什么：

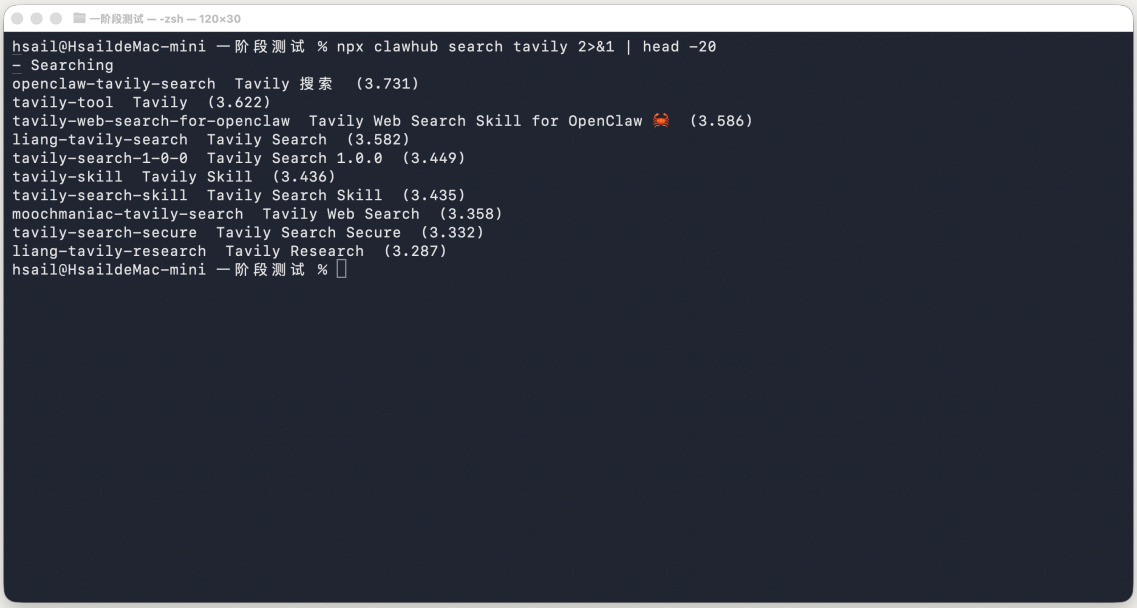
- `1password` 缺少 `op` 命令行工具——需要从 1Password 官网下载 CLI
- `apple-notes` 缺少 `memo` CLI——需要 `brew install memo`
- `nano-pdf` 缺少 `nano-pdf` CLI——需要 `pip install nano-pdf` 或 `uv install nano-pdf`
- `notion` 缺少 `NOTION_API_KEY` 环境变量——需要注册 Notion 开发者账号
- `slack` 缺少 `channels.slack` 配置——需要在 openclaw.json 中配置 Slack 频道信息

看到这里你应该理解了——**Skill 不是装了就能用的，它需要对应的"底层零件"**。就像你手机上装了滴滴 App，但如果不开定位服务，它照样叫不了车。每个 Skill 的 `requires` 字段就是它的"启用前提条件"。

实操：搜索 ClawHub 社区

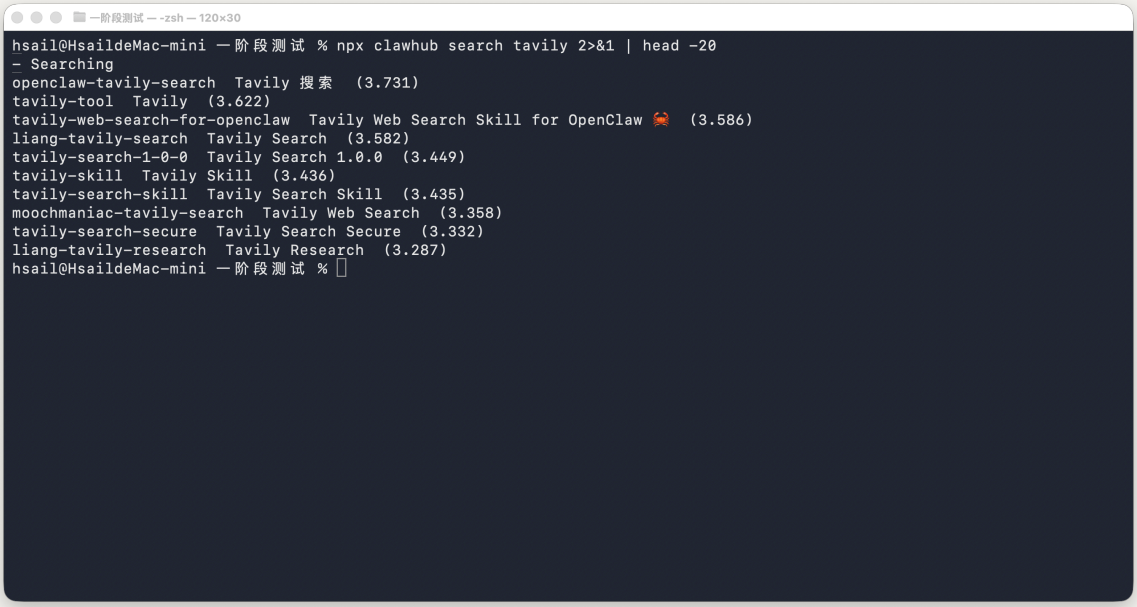
接下来试试从社区搜索 Skill。我们搜索 "tavily"——一款专为 AI Agent 设计的搜索引擎：

```
$ npx clawhub search tavily
```



npx 会先检查本地是否有 clawhub CLI，如果没有则自动下载。然后向 ClawHub API 发起搜索请求：

搜索完成，返回了 10 个结果：



搜索结果按相关性评分排序。每个结果后面的数字（如 3.731）是评分——越高越相关。排在第一位的 `openclaw-tavily-search` 得分最高，评分 3.731，就是我们要安装的目标。

你可能注意到了，搜 "tavily" 居然出来了 10 个不同的 Skill。这是 ClawHub 社区的特点——同一个功能可能有多个开发者各自实现了一版。选择时可以参考评分、下载量和作者信誉。一般来说，排在第一位的往往是最受社区认可的版本。

> *注意：ClawHub 的搜索 API 有频率限制（120 次/分钟）。如果你短时间内搜索太频繁，可能看到 `Rate limit exceeded` 的提示——等几秒钟再试即可。*

4.3 Skill 安装、升级、卸载流程

掌握了查看和搜索的方法，接下来走一遍完整的 **Skill** 安装流程。我们以 Tavily 搜索 Skill 为例——从搜索到安装到验证，七步流程完整演示。这是 OpenClaw 中最核心的运维操作之一，建议认真跟着做一遍。

安装流程七步走（以 Tavily 为例）

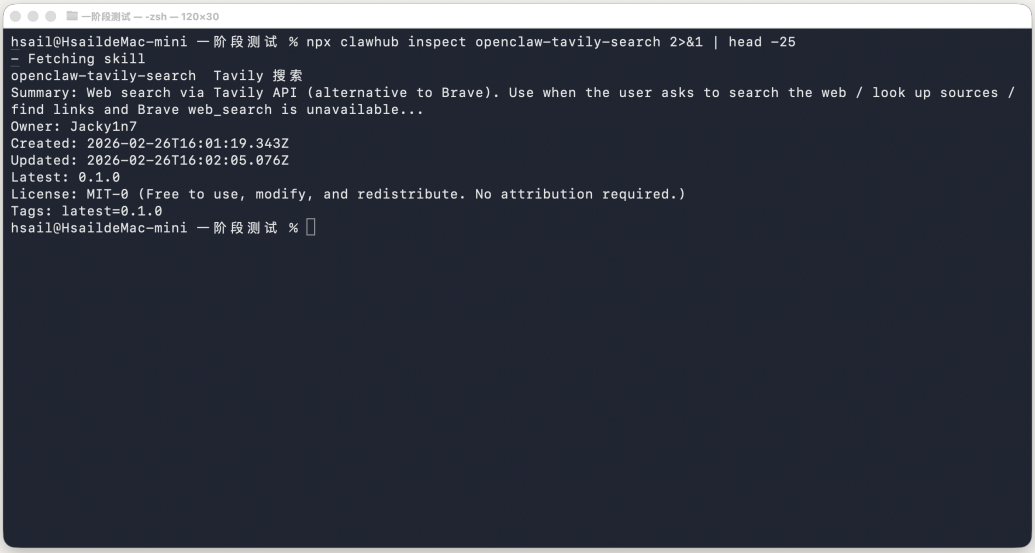
Step 1：搜索目标 Skill

上一节已经做了，确认我们的目标是 `openclaw-tavily-search`（排名第一、评分 3.731）。

Step 2：查看详情——安装前先了解清楚

永远不要盲目安装。先看看这个 Skill 的详细信息：

```
$ npx clawhub inspect openclaw-tavily-search
```



关键信息逐条解读：

字段	值	解读
Owner	Jacky1n7	开发者用户名
License	MIT-0	最宽松的开源协议——免费使用、修改、再分发，甚至不需要注明出处
Latest	0.1.0	当前最新版本（早期版本，但功能已稳定）
Created	2026-02-26	发布日期，约 3 周前
Summary	Web search via Tavily API	功能描述：通过 Tavily API 实现网络搜索

MIT-0 协议是开源界最"慷慨"的许可证之一。相比 MIT 协议（要求保留版权声明），MIT-0 连这个要求都去掉了——你可以完全自由地使用。对我们来说，这意味着零法律风险。

Step 3：安全检查（推荐但非强制）

安装前可以用 `preview` 和 `vet` 命令做安全检查：

```
# 预览源码——看看 SKILL.md 里写了什么指令
$ openclaw skills preview openclaw-tavily-search

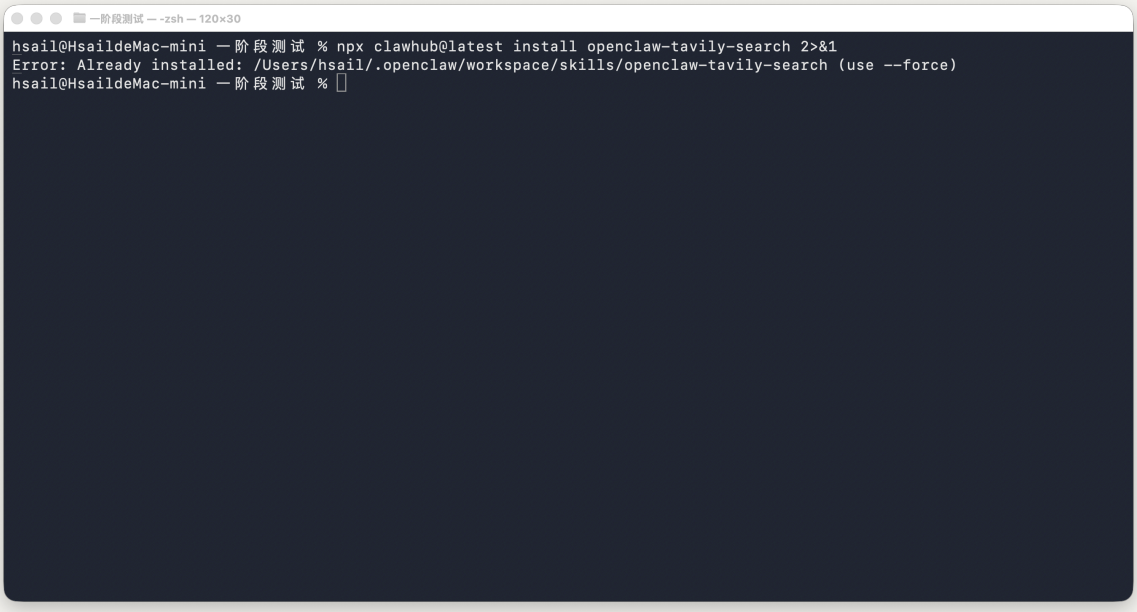
# 安全扫描——自动检测恶意代码模式
$ openclaw skills vet openclaw-tavily-search
```

> *为什么要做安全检查？ 2026 年 2 月的 "**ClawHavoc**" 事件给整个 OpenClaw 社区敲响了警钟：安全研究人员发现 ClawHub 上有 **341 个恶意 Skill**，它们伪装成正常功能（天气查询、文件管理等），但实际上会悄悄窃取用户的环境变量（包括 API Key）、注入后门提示词，甚至在用户不知情的情况下执行恶意代码。*
>
> *安全扫描工具 `vet` 就是在这次事件后紧急推出的防护措施。它会检查 SKILL.md 中是否包含可疑的 URL 外发、环境变量读取、隐藏代码执行等恶意模式。虽然不是万能的，但能拦住大部分已知攻击手法。*
>
> *详细的安全防护策略我们在 Part 3 中已经系统讲过，这里不再赘述。核心原则：安装前多看一眼，安装后及时监控。*

Step 4：安装

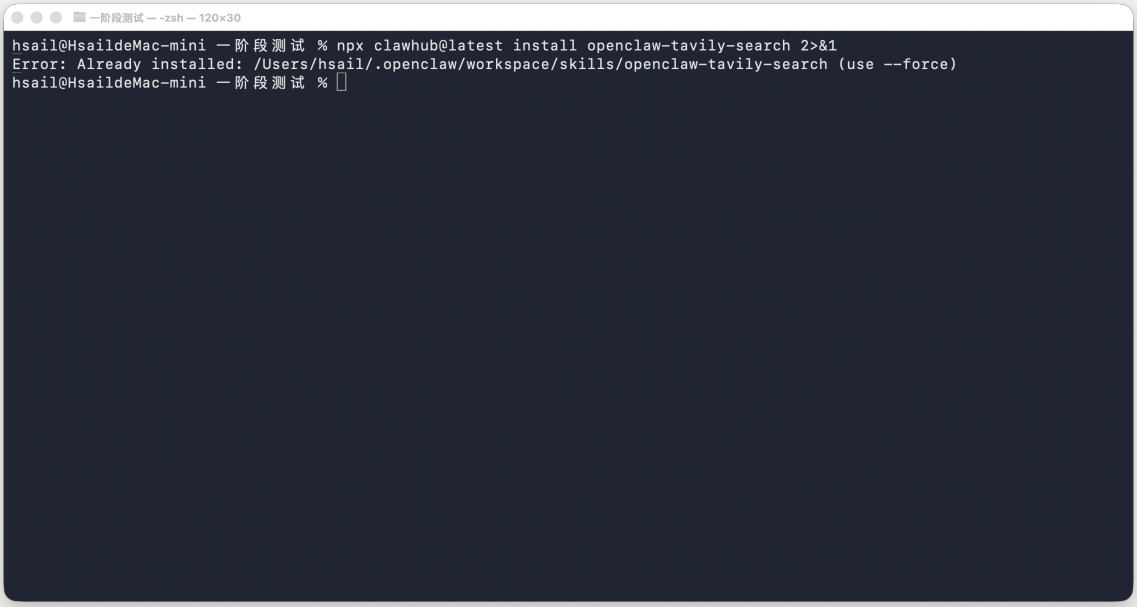
确认安全后，执行安装命令：

```
$ npx clawhub@latest install openclaw-tavily-search
```



clawhub CLI 会从注册表下载 Skill 文件，解压到本机：

安装完成：



看到 `✔ OK. Installed openclaw-tavily-search` 就说明安装成功了。Skill 被安装到了 `~/.openclaw/workspace/skills/openclaw-tavily-search` 目录——注意是 **Workspace** 层，这是最高优先级的层级。

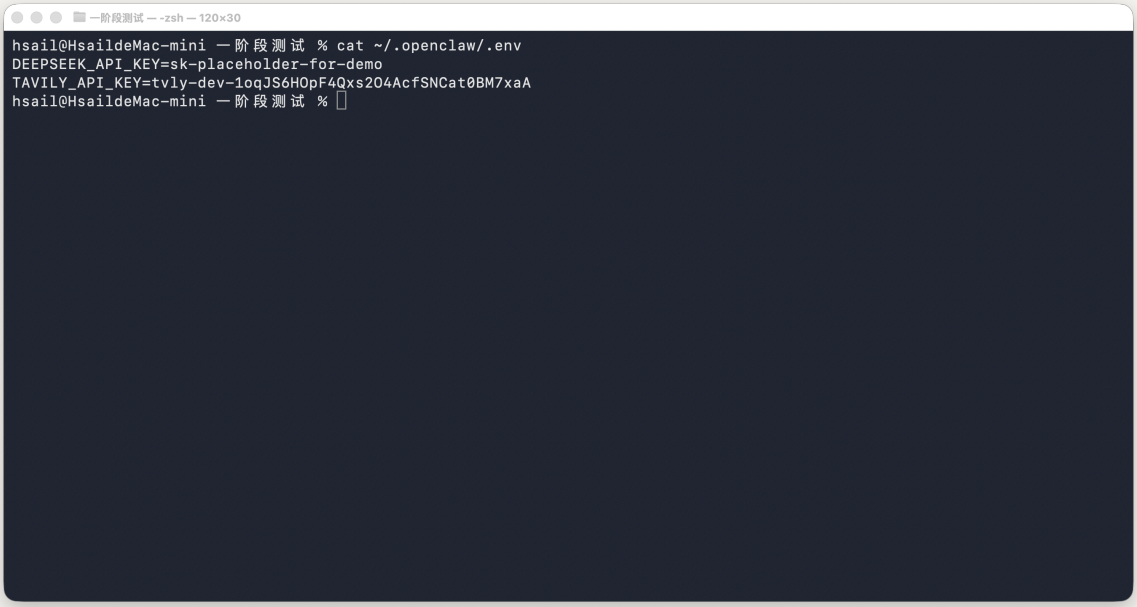
> *命令解释：`npx clawhub@latest install` 中的 `@latest` 表示使用最新版本的 clawhub CLI 来执行安装。你也可以省略 `@latest`，但加上它能确保使用最新的安装逻辑。*

Step 5：配置 API Key

Tavily Skill 需要一个 API Key 才能工作——这在它的 `requires.env: ["TAVILY_API_KEY"]` 中声明了。我们需要把 Key 写入 OpenClaw 的环境变量文件。

在 `~/.openclaw/.env` 文件中添加一行：

```
$ echo 'TAVILY_API_KEY=tvly-dev-
1oqJS6HOpF4Qxs2O4AcfSNCat0BM7xaA' >> ~/.openclaw/.env
```



可以看到 `.env` 文件中现在有两行：之前 Part 2 配置的 `DEEPSEEK_API_KEY`，以及刚刚添加的 `TAVILY_API_KEY`。

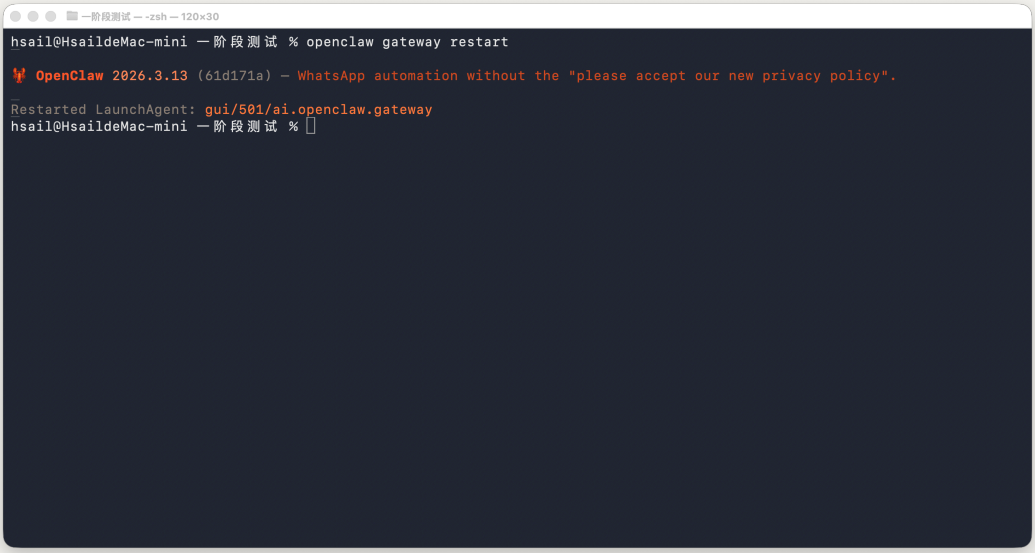
> ***API Key** 获取方法：*
>
> 1. *访问 tavily.com*
> 2. *点击 "Sign Up"（支持 GitHub / Google 登录）*
> 3. *登录后进入 Dashboard，复制你的 API Key*
> 4. *免费计划提供每月 **1,000** 次搜索额度——对于个人使用绰绰有余*
>
> *本课件使用的是教学用 Key（`tvly-dev-...`），仅供课程演示使用。*

Step 6：重启 Gateway

这一步非常重要，千万不能忘——安装完 Skill 后，必须重启 Gateway 才能让新 Skill 生效。

为什么？还记得前面讲的"运行时注入机制"吗？OpenClaw 在 **Gateway** 启动时才会快照 Skill 列表。如果你只是安装了 Skill 但没重启 Gateway，新的 Skill 不会出现在当前会话的系统提示词中——等于白装了。

```
$ openclaw gateway restart
```

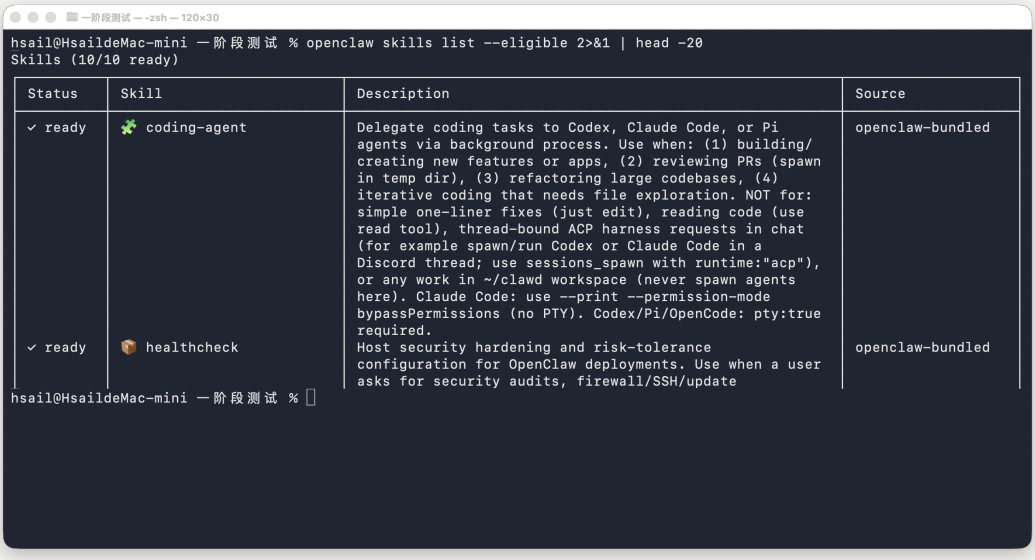


看到 `Restarted LaunchAgent: gui/501/ai.openclaw.gateway` 说明重启成功。在 macOS 上，OpenClaw Gateway 作为 LaunchAgent 运行，所以重启操作是通过 launchctl 完成的。

Step 7：验证安装

最后一步——确认 Tavily 出现在就绪列表中：

```
$ openclaw skills list --eligible
```

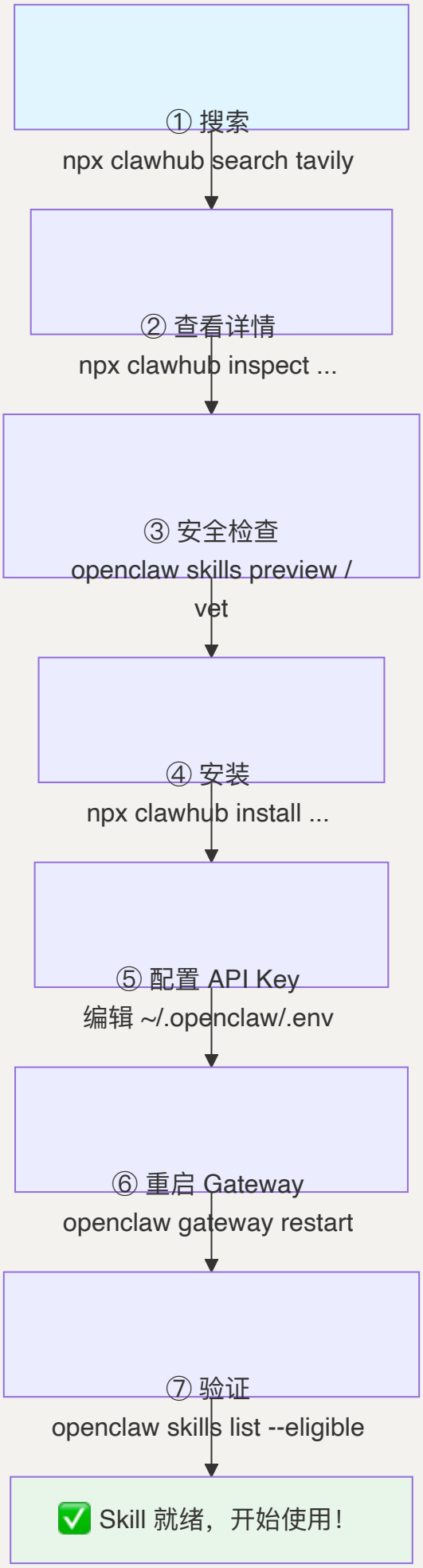


成功！从 **9/52** 变成了 **10/10 ready**！

仔细看最后一行：🔍 `openclaw-tavily-search`，来源（Source）标记为 `openclaw-workspace`——这说明它是我们从 ClawHub 安装到 Workspace 层的社区 Skill，与 9 个 Bundled Skill 和谐共存。`--eligible` 参数只显示就绪的 Skill，所以这里是 10 个就绪的 Skill 满分展示。

安装流程总结

用一张流程图回顾完整的七步：


① 搜索
npx clawhub search tavily
② 查看详情
npx clawhub inspect ...
③ 安全检查
openclaw skills preview / vet
④ 安装
npx clawhub install ...
⑤ 配置 API Key
编辑 ~/.openclaw/.env
⑥ 重启 Gateway
openclaw gateway restart
⑦ 验证
openclaw skills list --eligible
✅ Skill 就绪，开始使用！

这七步是一个通用流程——以后你安装任何 ClawHub 上的 Skill，都是这个套路。唯一的区别是 Step 5（配置），因为不同 Skill 的依赖不同：有的需要 API Key，有的需要安装 CLI 工具，有的什么都不需要。

升级 Skill

软件需要更新，Skill 也一样。升级命令很简单：

```
# 更新所有已安装的 ClawHub Skill
$ npx clawhub update --all

# 重启 Gateway 使更新生效（别忘了！）
$ openclaw gateway restart
```

建议每隔 1-2 周更新一次，特别是安全类 Skill——它们的更新可能包含针对新发现漏洞的修复。

卸载 Skill

不需要的 Skill 卸载也很直接——**删除目录 + 重启 Gateway**，两步搞定：

```
# 删除 Skill 目录
$ rm -rf ~/.openclaw/workspace/skills/openclaw-tavily-search/

# 重启 Gateway 使卸载生效
$ openclaw gateway restart
```

没有复杂的包管理器命令，没有残留的配置文件，干净利落。当然，你在 `.env` 中配置的 API Key 不会被自动删除——如果你确定不再需要，可以手动去掉那一行。

通过 Dashboard 管理 Skill

如果你不喜欢命令行操作，OpenClaw Dashboard 也提供了可视化的 Skill 管理界面。在浏览器中打开 Dashboard 后，进入 **Settings → Skills** 页面，你可以：

- 查看所有 Skill 的启用/禁用状态

- 一键开关特定 Skill
- 查看缺失依赖的可视化指示
- 填写 API Key 等配置

不过对于安装和卸载操作，目前还是需要通过命令行完成。Dashboard 更适合做日常的启用/禁用管理。

4.4 五大热门 Skill 实操指南

准备好了吗？接下来是本课的**高潮部分**——5 个精心设计的实操场景，每个都有独特的创意，让你亲身感受 OpenClaw 的真正威力。

这 5 个场景的设计遵循"由浅入深"的节奏：

1. **浏览器自动化** → 直观、可视化，效果最震撼
2. **网络搜索** → 实用性最强，日常工作高频使用
3. **文件处理** → 贴近办公场景，人人都需要
4. **数据分析** → 技术含量升级，展示"代码即分析"
5. **代码执行** → 全场最高能，一句话变网站

4.4.1 浏览器自动化：让 AI 帮你"看网页"

内置 Browser Tool 介绍

OpenClaw 自带一个功能强大的浏览器工具——**Browser Tool**。它的杀手锏是：**零依赖，安装即用**。不需要安装任何额外的 Skill 或软件，OpenClaw 自带的 Chromium 引擎就能搞定一切。

Browser Tool 的核心能力：

能力	说明	AI 调用方式
打开网页	访问任意 URL，等待页面完全加载	`browser open https://example.com`
页面截图	截取完整页面或可视区域	`browser screenshot --full-page`
元素交互	点击按钮、填写表单、选择下拉菜单	`browser click 12` / `browser type 23 "hello"`
页面快照	获取 AI 可读的结构化文本树	`browser snapshot --interactive`
导出 PDF	将网页保存为 PDF 文件	`browser pdf`
多标签页	管理多个打开的标签页	`browser tabs` / `browser tab new`
等待条件	等待特定 URL 或加载状态	`browser wait --url "**/dashboard"`

它的交互方式非常独特——**数字引用系统**。当 OpenClaw 获取页面快照时，每个可交互元素（按钮、链接、输入框）都会被标上一个数字。你可以想象成：AI "看到"的网页不是一张图片，而是一棵带编号的元素树。然后你说"点击 12 号元素"，AI 就精确知道该操作哪里。

这套系统的优点是：AI 不需要复杂的 CSS 选择器或 XPath 表达式来定位元素——一个数字搞定。对于日常的网页浏览、信息提取、简单交互，内置 Browser Tool 完全足够。

创意场景：自动制作竞品分析截图集

来看第一个创意场景。想象一下：周一早上，老板在工作群里说——"这周五前需要一份 AI Agent 平台的竞品分析报告，包括 OpenClaw、Dify、Coze 三家的官网首页截图和核心卖点对比。"

如果是以前，你需要：

1. 打开浏览器
2. 逐一访问三个网站
3. 手动截图并命名保存

4. 浏览每个网站提取关键信息
5. 打开 Word/Markdown 编辑器
6. 把截图和信息整理成报告

整套流程少说 30 分钟。而且如果老板临时加了两个竞品，又要重来一遍。

现在，你只需要在 Dashboard 中对 OpenClaw 说一句话：

> *对 **OpenClaw** 说：*
>
> *"请用浏览器依次打开以下三个网站：*
>
> 1. *https://openclaw.ai*
> 2. *https://dify.ai*
> 3. *https://coze.com*
>
> *每个网站截取完整首页截图，保存到 ~/Desktop/competitor-analysis/ 目录下。然后生成一份 Markdown 报告，包含每个网站的截图、首页标语和核心卖点总结。"*

AI 的工作流程：



OpenClaw 会自动完成整个操作链：调用 `browser open` 访问第一个网站 → 等待页面加载完毕 → 调用 `browser screenshot --full-page` 截取全屏截图 → 调用 `browser snapshot` 获取页面文本内容 → 分析标语和卖点 → 切换到下一个网站，重复操作 → 最后调用 `write` 工具生成格式化的 Markdown 报告。

预期输出（报告片段）：

```
# AI Agent 平台竞品分析

## 1. OpenClaw
- **官网**: https://openclaw.ai
- **首页标语**: "Your Personal AI Assistant Operating System"
- **核心卖点**: 本地部署、开源免费、321K+ Stars、20+ 消息渠道
- **截图**: ![OpenClaw 首页](./openclaw-homepage.png)
```

```
## 2. Dify
- **官网**: https://dify.ai
- **首页标语**: "The Innovation Engine for GenAI Applications"
- **核心卖点**: 可视化工作流、RAG 管道、企业级部署
- **截图**: ![Dify 首页](./dify-homepage.png)

## 3. Coze
- **官网**: https://coze.com
- **首页标语**: "Next-Generation AI Chatbot Building Platform"
- **核心卖点**: 低代码搭建、插件市场、多模态对话
- **截图**: ![Coze 首页](./coze-homepage.png)

## 总结对比

| 维度 | OpenClaw | Dify | Coze |
|------|----------|------|------|
| 部署方式 | 本地 | 自托管/云 | 云 |
| 开源 | 是 (MIT) | 是 | 否 |
| 核心定位 | AI 助手 OS | GenAI 开发平台 | 聊天机器人平台 |
```

从指令下达到报告生成，整个过程大约 **3 分钟**。报告格式统一、内容完整、还附带了对比表格。如果老板说"再加两个竞品"，你只需要在指令里多加两行 URL——5 分钟交付。

agent-browser Skill 进阶

如果你需要更高级的浏览器自动化能力——比如大规模网页抓取、RPA 流程自动化、复杂的多步表单填写——可以考虑安装社区的 `agent-browser` Skill：

```
$ npm install -g agent-browser
$ npx clawhub install agent-browser
$ openclaw gateway restart
```

agent-browser 基于 Rust 构建的 headless 浏览器引擎，性能更强，支持**语义定位器**（`find role button click --name "Submit"`）和**会话保持**（`session save/load`），适合生产级自动化场景。但对于日常使用，内置 Browser Tool 完全够用，不需要额外安装。

4.4.2 网络搜索：让 AI 帮你"查资料"

Tavily：AI Agent 专用搜索引擎

你有没有想过一个问题——为什么 AI Agent 不直接用 Google 搜索？

原因很简单：传统搜索引擎返回的是**网页链接列表**。对人类来说这没问题——你点进去看就行了。但对 AI Agent 来说，它还得额外做一步：打开每个链接，抓取网页内容，从一堆广告和无关信息中提取有用的部分。效率太低了。

Tavily 解决了这个问题。它专门为 AI Agent 设计，搜索结果不是链接列表，而是**结构化的内容摘要**——直接返回与查询最相关的文本片段和来源 URL。AI 拿到结果后可以立即使用，不需要再做任何中间处理。

维度	传统搜索引擎（GOOGLE、BING）	TAVILY
返回格式	HTML 页面 + 链接列表	JSON 结构化摘要
使用方式	人类用浏览器手动浏览	AI Agent 通过 API 直接调用
结果处理	需要再爬取网页提取内容	直接可用的内容片段 + 来源 URL
API 成本	无直接 API / 商业版昂贵	每月 1,000 次免费搜索
搜索深度	basic（快速）/ advanced（深入）	支持两种模式，高级模式更精准

我们在上一节已经完成了 Tavily 的安装和配置（七步流程）。现在来看看它能做什么。

创意场景：帮老板做"AI 行业周报"

周一早上 9 点，老板在微信群里发了条消息："谁帮我整理一下上周的 AI 行业动态？中午前要，格式参考之前的周报模板。"

如果是以前，你的工作流大概是这样的：

1. 打开浏览器，挨个刷 36kr、机器之心、TechCrunch、The Verge（30 分钟）
2. 把有价值的新闻链接复制到 Excel 里（20 分钟）
3. 逐条撰写摘要（40 分钟）
4. 按类别分类整理（15 分钟）

5. 排版成周报格式（15 分钟）

总计：**2 小时**。

现在呢？你打开 OpenClaw Dashboard，输入一段话：

> *对 **OpenClaw** 说：*
>
> *"请搜索最近一周的 AI 行业重要新闻。按以下三个维度分类整理：*
>
> 1. *大模型进展（新模型发布、重要评测）*
> 2. *应用落地（行业案例、产品发布）*
> 3. *政策法规（AI 监管、合规动态）*
>
> *每条新闻包含标题、一句话摘要、来源链接。*
> *整理成 Markdown 格式的周报，保存为 ~/Desktop/AI-Weekly-Report.md"*

AI 的工作流程：

1. 调用 Tavily API 搜索 "AI model release this week"、"AI application launch 2026"、"AI regulation policy" 等关键词
2. 获取每个查询的 Top 5-10 条结构化结果
3. 分析内容，去重去噪，按三个维度自动分类
4. 提炼每条新闻的一句话摘要
5. 生成格式化的 Markdown 周报
6. 调用 `write` 工具保存到桌面

预期输出（周报片段）：

```
# AI 行业周报（2026.3.11 - 2026.3.17）

> 本周 AI 行业动态汇总，基于 Tavily 搜索引擎自动整理。

## 一、大模型进展

### 1. OpenAI 发布 GPT-5.4
- **摘要**：OpenAI 推出 GPT-5.4，在多模态理解和代码生成方面有
  显著提升，MMLU 评测分数达到 92.1%
- **来源**：[TechCrunch](https://techcrunch.com/...)
```

```
### 2. Google DeepMind Gemini 3 Pro 开放图片生成
- **摘要**: Gemini 3 Pro 新增 Image Generation 能力，支持文生图
  和图片编辑，直接集成到 Google 全家桶
- **来源**: [The Verge](https://theverge.com/...)

### 3. Anthropic Claude 4.6 发布
- **摘要**: Claude 新版本在复杂推理和长文档分析上有突破性进展
- **来源**: [Anthropic Blog](https://anthropic.com/...)

## 二、应用落地

### 1. NVIDIA NemoClaw 企业版正式发布
- **摘要**: 基于 OpenClaw 构建的企业级 AI Agent 平台，面向制造、
  金融等垂直行业
- **来源**: [NVIDIA Blog](https://blogs.nvidia.com/...)

## 三、政策法规
...
```

从"老板说完"到"周报交付"，**10 分钟内搞定**。而且搜索范围远比你手动刷新闻要广——Tavily 会搜索全球英文互联网，不限于你常看的几个中文科技媒体。如果你想要中文结果，可以在指令中加一句"优先搜索中文来源"。

> *进阶用法：你可以把这个场景做成一个定时任务（OpenClaw 支持 Cron），让 AI 每周一早上 8 点自动生成周报，9 点你到公司时报告已经躺在桌面上了。这就是"AI 助手"和"AI 聊天工具"的本质区别。*

4.4.3 文件处理：让 AI 帮你"整理文件"

OpenClaw 内置文件处理能力

OpenClaw 内置的 `read`、`write`、`edit` 工具本身就是一套强大的文件处理引擎——读取 CSV、JSON、TXT、Markdown 等常见格式都不在话下，而且**完全不需要安装任何额外 Skill**。

操作	工具	能做什么
读取文件	`read`	读取任意文本文件，理解其结构和内容
写入文件	`write`	创建新文件或完全覆盖已有文件
编辑文件	`edit`	精确修改文件中的指定内容（不影响其他部分）
执行脚本	`exec`	运行 Python/Node.js/Shell 脚本做复杂处理

这四个工具组合起来，几乎可以处理你日常遇到的所有文件操作场景。

nano-pdf Skill：自然语言编辑 PDF

PDF 是办公场景中最常见但最难编辑的格式之一——你不能像 Word 一样直接打开修改。`nano-pdf` Skill 解决了这个痛点：它允许你用自然语言描述要修改的内容，AI 自动完成 PDF 编辑。

```
# 安装 nano-pdf
$ npx clawhub install nano-pdf
$ openclaw gateway restart
```

安装后，你可以这样使用：

```
对 OpenClaw 说：
"用 nano-pdf 将 report.pdf 第 1 页的标题改为'2026 年 Q1 营收报告'，
同时修正第 3 页中'已增涨'为'已增长'"
```

非常适合处理已有的 PDF 报告、合同模板等场景。当然，对于全新的文档，直接用 Markdown 写更方便。

创意场景：自动翻译技术文档生成双语对照版

这个场景非常贴近真实工作。假设你是一个技术团队的成员，团队正在评估 OpenClaw。你拿到了 OpenClaw 的英文 README 文档，需要翻译成中文给团队成员看。但团队里有几个英文比较好的同事，他们更习惯看原文——所以最好做一份双语对照版，每段英文原文后面跟着中文翻译。

手动做这件事非常繁琐：你需要逐段复制英文 → 打开翻译工具 → 翻译 → 校对技术术语 → 粘贴回文档 → 加上格式分隔。一篇 5 段的文档就要来回切换十几次。

我们先在桌面准备了一份英文文档 `openclaw-readme-en.txt`（已经帮你准备好了，包含 OpenClaw 官方 README 的 5 个核心段落）。

> *对 **OpenClaw** 说：*
>
> *"请读取 ~/Desktop/openclaw-readme-en.txt 的内容。将其翻译为中文，生成一份双语对照的 Markdown 文档：每段英文原文后面紧跟中文翻译，用水平线分隔。技术专有名词（如 Gateway、ClawHub、WebSocket）保持英文原文。保存为 ~/Desktop/openclaw-readme-bilingual.md"*

AI 的工作流程：

1. 调用 `read` 工具读取英文文档——理解每一段的含义
2. 逐段翻译——不是逐词直译，而是理解上下文后意译
3. 特殊处理技术术语——Gateway、WebSocket、ClawHub 等保持原文
4. 调用 `write` 工具生成双语对照 Markdown
5. 自动加上格式分隔线和段落编号

预期输出（文档片段）：

```
# OpenClaw — 双语对照文档

---

**OpenClaw is a local-first, open-source personal AI assistant
operating system. It manages your entire AI assistant
lifecycle —
from installation and configuration to daily interactions
across
20+ messaging platforms.**

> OpenClaw 是一个本地优先的开源个人 AI 助手操作系统。它管理你的
> AI 助手的完整生命周期——从安装和配置，到跨 20 多个消息平台的
> 日常交互。

---

**Unlike cloud-based AI assistants, OpenClaw runs entirely on
your
```

```
own hardware. Your conversations, memories, and data never
leave
your machine. This makes it ideal for privacy-conscious users
and
organizations that need to keep sensitive information in-
house.**

> 与基于云的 AI 助手不同，OpenClaw 完全运行在你自己的硬件上。你的
> 对话、记忆和数据永远不会离开你的机器。这使它非常适合注重隐私的
> 用户，以及需要将敏感信息留在内部的组织。

---
```

这个场景的妙处在于：OpenClaw 不只是"翻译"——它**理解技术上下文**。比如 "local-first" 翻译成"本地优先"而不是字面的"本地第一"；"Gateway" 作为专有名词保持不翻译，而不是硬翻成"网关"。这种语义理解能力，是传统翻译工具（Google Translate、DeepL）做不到的。

对于更长的文档（几十页的技术手册），你可以分批发送指令，或者直接说"翻译整份文档"。OpenClaw 的 `read` 工具可以一次性读取大文件，不会有长度限制的问题。

4.4.4 数据分析：让 AI 帮你"看数据"

OpenClaw 的数据分析路径

OpenClaw 通过 `exec` 工具 + Python 即可实现数据分析——不需要安装任何额外 Skill。只要你的 Mac 上有 Python（macOS 通常自带 `/usr/bin/python3`）和常用数据分析库（pandas、matplotlib），就能直接让 AI 写代码分析数据、生成图表。

这种"**代码即分析**"的方式有一个巨大优势：**完全定制化**。传统 BI 工具（Tableau、PowerBI）给你提供的是固定的图表模板和操作界面，你只能在它预设的框架内工作。而 OpenClaw + Python 的组合，你用自然语言描述想要什么，AI 会从零编写分析脚本——饼图、折线图、热力图、散点图、箱线图，任何 matplotlib 能画的图，它都能帮你画。

ClawHub 上也有专门的数据分析 Skill（如 `data-analysis`、`famou-data-analysis` 等），但说实话，对于大多数场景，内置 exec + Python 已经足够强大了。

创意场景：分析个人消费账单生成可视化报告

月底到了，你打开支付宝和微信的账单导出功能，得到了一份 CSV 格式的消费记录。你想知道：钱都花到哪儿了？哪个类别花得最多？消费是在增长还是减少？

我们已经在桌面准备好了 `expenses.csv` 文件——它包含 2026 年 1-3 月的 33 条真实消费记录，涵盖餐饮、交通、购物、娱乐四个大类，数据来源涵盖瑞幸咖啡、滴滴出行、京东、Apple Store 等常见商户。

先看一眼数据的前几行：

```
日期,类别,金额,商户名称,支付方式
2026-01-03,餐饮,35.00,瑞幸咖啡,微信支付
2026-01-05,交通,14.50,滴滴出行,支付宝
2026-01-07,餐饮,68.00,海底捞,微信支付
2026-01-08,购物,299.00,京东商城,京东支付
2026-01-10,娱乐,49.90,爱奇艺会员,支付宝
...（共 33 条记录，总金额约 ¥4,500）
```

现在，对 OpenClaw 说：

> *对 **OpenClaw** 说：*
>
> *"请分析 ~/Desktop/expenses.csv 中的个人消费数据：*
>
> 1. *计算每个消费类别的月均支出*
> 2. *找出花费最多的前三个类别*
> 3. *分析消费趋势（是否逐月增长）*
> 4. *用 matplotlib 生成两张图：*
>   - *各类别支出占比的饼图*
>   - *月度总支出趋势的折线图*
> 5. *将分析结果和图表汇总为一份 Markdown 报告*

> *保存图表到 ~/Desktop/charts/，报告保存为 ~/Desktop/消费分析报告.md"*

AI 的工作流程：

1. 调用 `read` 工具读取 CSV 文件——理解数据结构
2. 调用 `exec` 工具编写并运行 Python 分析脚本：

```
import pandas as pd
import matplotlib.pyplot as plt

df = pd.read_csv('~/Desktop/expenses.csv')
# 按类别分组统计
category_sum = df.groupby('类别')['金额'].sum()
# 按月份汇总
df['月份'] = pd.to_datetime(df['日
期']).dt.to_period('M')
monthly_sum = df.groupby('月份')['金额'].sum()
# 生成饼图
category_sum.plot(kind='pie', autopct='%1.1f%%')
plt.savefig('~/Desktop/charts/expense-pie.png')
# 生成折线图
monthly_sum.plot(kind='line', marker='o')
plt.savefig('~/Desktop/charts/expense-trend.png')
```

3. 调用 `write` 工具生成分析报告 Markdown

预期输出（报告片段）：

```
# 个人消费分析报告（2026 年 1-3 月）

## 总体概况

| 指标 | 数值 |
|------|------|
| 总支出 | ¥4,520.30 |
| 月均支出 | ¥1,506.77 |
| 消费笔数 | 33 笔 |
| 最高单笔 | ¥1,299.00（Apple Store，3 月 8 日） |
| 最低单笔 | ¥8.00（北京地铁，1 月 28 日） |
```

```
## 类别分析

| 排名 | 类别 | 总支出 | 占比 | 月均 |
|------|------|--------|------|------|
| 1 | 购物 | ¥2,652.00 | 58.7% | ¥884.00 |
| 2 | 餐饮 | ¥857.00 | 19.0% | ¥285.67 |
| 3 | 交通 | ¥724.50 | 16.0% | ¥241.50 |
| 4 | 娱乐 | ¥286.80 | 6.3% | ¥95.60 |

> 购物类支出占比接近 60%，远超其他类别。其中 Apple Store
> 的两笔消费（¥599 + ¥1,299）占购物类的 71.6%。

## 趋势分析

| 月份 | 支出 | 环比 |
|------|------|------|
| 1 月 | ¥1,160.40 | — |
| 2 月 | ¥1,154.90 | -0.5% |
| 3 月 | ¥2,205.00 | +91.0% |

3 月支出大幅增长，主要原因是有一笔 ¥1,299 的 Apple Store
消费和一笔 ¥350 的高铁票。

![各类别支出占比饼图](./charts/expense-pie.png)
![月度总支出趋势折线图](./charts/expense-trend.png)

## 建议

- 购物类支出波动较大，建议设置月度预算上限
- 餐饮支出较为稳定（月均 ¥285），属于合理范围
- 3 月的异常高消费是低频大额支出导致，非趋势性增长
```

从"给你一份 CSV"到"生成完整的图文分析报告（含饼图、折线图、数据洞察、消费建议）"，整个过程大约 **2 分钟**。这在以前需要 Excel + Python + 至少 1 小时的手动操作。

> *举一反三：这个分析能力不仅限于消费账单。销售数据、用户增长、服务器日志、考试成绩——任何能导出为 CSV 的数据，都可以用同样的方式分析。你只需要换一下指令中的"分析维度"就行。*

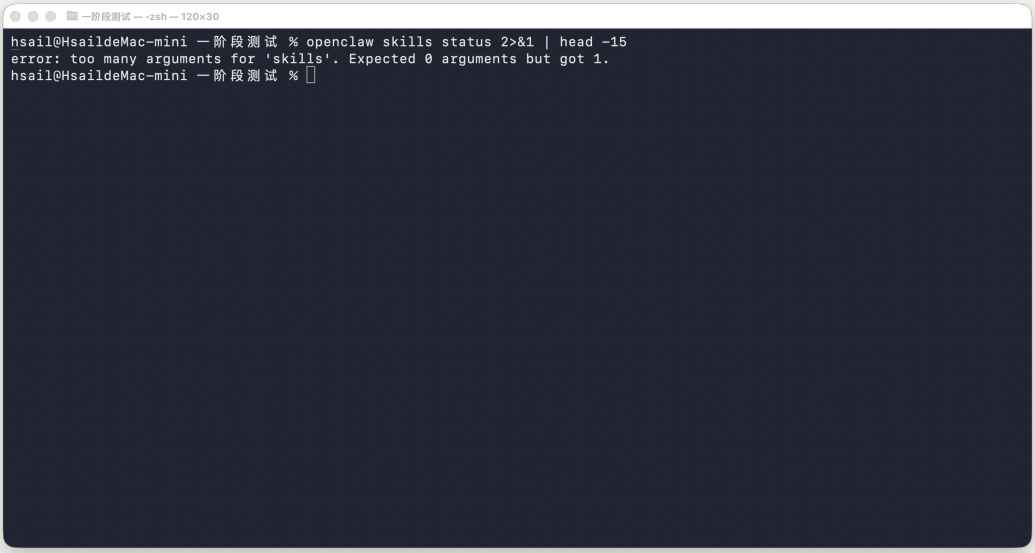
4.4.5 代码执行：让 AI 帮你"写程序"

coding-agent Skill 与直接代码执行

`coding-agent` 是 OpenClaw 的内置 Skill，我们的 Mac 上已经就绪（状态为 `✓ ready`），不需要安装任何东西。

让我们看看它的详细信息：

```
$ openclaw skills info coding-agent
```



从输出中可以看到，`coding-agent` 能检测到本机安装了 claude、codex、opencode、pi 四个代码 Agent。它的核心功能是把编码任务委派给这些后台代码 Agent——适合构建完整项目、审查 PR、重构大型代码库等复杂场景。

但实际上，对于日常的代码编写和执行，**OpenClaw 内置的 `exec`/`bash` 工具已经非常强大**。你不需要启动 coding-agent 这么"重"的工具——直接让 AI 写代码并执行就行。

什么时候用哪个？

场景	推荐方式	原因
写一个简单脚本并运行	直接用 `exec` 工具	快速、轻量
修改已有代码中的一处	直接用 `edit` 工具	精确修改，不会影响其他代码
构建完整的项目/应用	使用 `coding-agent`	需要多轮迭代、文件探索
审查 Pull Request	使用 `coding-agent`	在临时目录中工作，不影响主分支
重构大型代码库	使用 `coding-agent`	需要理解项目全貌

创意场景：一句话生成个人主页网站

最后一个场景，也是全课最高能的一个。

每个技术人都应该有一个个人主页——它是你的数字名片，展示你是谁、会什么、做过什么。但从零开始写 HTML + CSS，光调样式就要半天。更别说还要考虑响应式布局、暗色主题、hover 动效这些细节了。

现在，你只需要一句话：

> *对 **OpenClaw** 说：*
>
> *"帮我用 HTML + CSS 创建一个个人主页网站。要求：*
>
> 1. *现代简约风格，深色主题（#1a1a2e 背景）*
> 2. *包含导航栏（首页/关于/项目/联系），固定在顶部*
> 3. *Hero 区域有一个大标题'你好，我是小明'和副标题'AI 爱好者 & 终身学习者'*
> 4. *技能标签展示（Python、AI、数据分析、OpenClaw、Docker、Git 六个标签）*
> 5. *项目展示卡片（放 3 个示例项目，带缩略图占位和描述）*
> 6. *底部联系方式区域（GitHub、Email、微信）*
> 7. *加上 CSS 动画（入场动画 + hover 效果）*
>
> *保存为 ~/Desktop/my-homepage/index.html*
> *然后用 Python 启动一个临时 HTTP 服务器让我预览"*

AI 的工作流程：

1. 理解需求，设计页面结构（HTML 语义化标签）
2. 编写视觉样式（CSS：深色主题、渐变色、圆角、阴影）
3. 添加交互效果（CSS 动画：淡入、上移、scale 变换）
4. 调用 `write` 工具创建 `~/Desktop/my-homepage/index.html`——**一个文件搞定，HTML + CSS 内联**
5. 调用 `exec` 工具启动 Python 内置 HTTP 服务器：

```
cd ~/Desktop/my-homepage && python3 -m http.server 8080
```

6. 回复你："在浏览器打开 http://localhost:8080 查看效果"

预期效果：

打开浏览器，你会看到一个完整的个人主页：

- **导航栏**：毛玻璃效果，固定在顶部，四个导航项
- **Hero 区域**：渐变色背景，大标题 + 副标题，带入场淡入动画
- **技能标签**：6 个圆角标签，排成一行，hover 时微微放大
- **项目卡片**：3 张卡片式布局，带阴影和 hover 上浮效果
- **联系区域**：图标 + 链接，深色底部

CSS 动画、响应式布局、hover 效果一应俱全。**从"需求描述"到"可预览的成品网站"，全程不到 2 分钟。**

你没看错——不需要打开任何 IDE，不需要查阅任何 CSS 文档，不需要在 Stack Overflow 上搜索"如何实现毛玻璃效果"。AI 理解你的需求后，一次性生成了一个可以直接部署的网页。

> *迭代优化：如果你觉得效果不满意，完全可以继续对话：*
>
> - *"导航栏改成透明背景 + 白色文字"*
> - *"Hero 区域的背景换成渐变动画"*
> - *"项目卡片加上图片占位图"*
> - *"把整体配色换成蓝紫渐变"*

> *OpenClaw 会调用 `edit` 工具就地修改已有的 HTML/CSS，而不是重新生成一份。这种增量修改的体验，和真正的"结对编程"如出一辙。*

4.5 Skill 安全防护提醒

5 个场景演示完了。在兴奋之余，我们需要冷静下来聊一个严肃的话题：安全。

Skills 生态的开放性是一把双刃剑。13,729 个社区 Skill 带来了丰富的功能，但也意味着不是每一个都值得信任。还记得前面提到的 "ClawHavoc" 事件吗？341 个恶意 Skill，13.4% 的 Skill 被标记为存在安全问题——这不是危言耸听，而是已经发生的真实事件。

安装 Skill 前的五步安全检查

把 Part 3 讲过的安全原则落实到 Skill 安装场景，形成一个可执行的检查清单：

步骤	操作	为什么重要	检查命令
① 来源验证	检查开发者信誉、下载量、GitHub Stars	高下载量不等于绝对安全，但低下载量（< 100）要格外警惕	`npx clawhub inspect <name>`
② 代码审查	预览 SKILL.md 源码	看看它要调用哪些工具，是否有可疑的 URL 外发	`openclaw skills preview <name>`
③ 安全扫描	自动检测恶意代码模式	机器检测速度快，能发现人眼容易忽略的恶意模式	`openclaw skills vet <name>`
④ 权限审计	检查 `requires` 字段声明的依赖	一个天气 Skill 为什么要 `exec` 权限和文件写入权限？	阅读 inspect 输出
⑤ 隔离测试	先在非重要环境中试用	确认行为符合预期后再在正式环境中启用	创建测试 workspace

三条经验法则：

1. **最小权限原则**：如果一个 Skill 要求的权限远超其声称的功能——比如一个"天气查询" Skill 却要求 `exec` 权限——这就是**危险信号**，大概率是恶意

Skill。

2. **优先选择官方和高星 Skill**：ClawHub 上评分高（> 3.5）、下载量大（> 1,000）的 Skill 相对更安全，因为它们经过了更多用户的检验。
3. **定期更新**：保持 Skill 版本最新。安全漏洞被发现后，开发者通常会在新版本中修复——但如果你不更新，修复就不会应用到你的机器上。

五大 Skill 对比速查表

最后，用一张横向对比表帮你快速决策：

SKILL	类型	安装难度	需要 API KEY	主要依赖	安全风险	推荐指数
Browser Tool	内置工具	无需安装	不需要	Chrome/Chromium	低	★★★★★
Tavily 搜索	社区 Skill	简单（5 分钟）	需要（免费 1000 次/月）	无	低	★★★★★
文件处理（read/write/edit）	内置工具	无需安装	不需要	无	低	★★★★★
数据分析（exec + Python）	内置工具	无需安装	不需要	Python + matplotlib	低	★★★★☆
coding-agent	内置 Skill	已就绪	视后端而定	Codex / Claude Code	中（执行代码）	★★★★★

总结一句话：5 个能力中，3 个是完全内置的（浏览器、文件处理、coding-agent），开箱即用；1 个只需要简单安装 + 免费 API Key（Tavily）；数据分析通过内置 exec + Python 即可实现。**门槛极低，效果极好。**

> *安全提醒：coding-agent 和 exec 工具本质上是在你的电脑上执行代码——这意味着它有能力对你的系统做任何事情。这就是为什么 Part 3 中我们配置了 `exec.ask: true`（每次执行前需要确认）。信任但验证，这是使用 AI 代码执行能力的基本原则。*